2026학년도 2학기 언어데이터과학

제10강 NSMC 데이터 활용: 형태소 분석과 단어구름

박수민

서울대학교 인문대학 언어학과

2026년 10월 7일 수요일

지난 시간에 배운 것

1. 글자 하나는 코드 포인트 하나다. UTF-8은 한글 한 글자를 3바이트로 적는다.
2. NFC와 NFD는 눈에는 같아도 기계에게는 다른 문자열이다.
3. 파일을 열 때는 `encoding=`을 언제나 적는다.

오늘의 목표

1. 어절과 형태소를 구별하고, 한국어를 공백으로 세면 안 되는 까닭을 말할 수 있다.
2. Kiwi로 문장을 형태소로 나누고 품사 태그를 읽을 수 있다.
3. 영화평 20만 편에서 품사를 골라 빈도를 셀 수 있다.
4. 단어구름을 그리고, 그 그림이 무엇을 감추는지 설명할 수 있다.

사전 준비

터미널 입력 (Codespaces) — 분석 모델 88MB를 함께 받으니 먼저 시작해 둔다

```
cd /workspaces/LDS2026
pip install kiwipiepy wordcloud
sudo apt-get update && sudo apt-get install -y fonts-nanum
```

노트북 10-nsmc.ipynb의 첫 셀

```
from unicodedata import name
import pandas as pd
from kiwipiepy import Kiwi
from wordcloud import WordCloud

kiwi = Kiwi()
```

fonts-nanum은 7강에서 미뤄 둔 한글 글꼴이다. 오늘 단어구름에 쓴다.

이 리뷰는 몇 단어인가

노트북 입력

```
s = '어릴때보고 지금다시봐도 재밌어요ㅋㅋ'
print(s.split())
print(len(s.split()))
```

실행 결과

```
['어릴때보고', '지금다시봐도', '재밌어요ㅋㅋ']
3
```

셋이라고 해도 될까

오늘 쓸 영화평 20만 편 가운데 첫 편이다. 공백으로 자르면 **셋**이다. 맞춤법대로 띄어 쓰면 **일곱**이 되고, 뜻을 가진 조각으로 나누면 **열둘**이 된다.

세 강의에 걸친 물음

강의	물음	답의 이름
8강	무엇을 하나로 셀 것인가	토큰, 타입, 문서
9강	글자 하나는 무엇인가	코드 포인트, 바이트
10강	**한국어에서 단어 하나는 무엇인가**	**어절, 형태소**

오늘은 새 데이터를 연다

3강부터 써 온 영어 요리책을 내려놓고, 처음으로 **큰 한국어 코퍼스**를 연다. 네이버 영화평 20만 편(NSMC)이다. 19강의 감정 분석에서 다시 쓴다.

8강 슬라이드에 적어 둔 예고가 이것이었다. “한국어는 공백만으로 단어를 셀 수 없다.”

1 **어절과 형태소**
공백이 자르는 것
형태소 분석

2 Kiwi로 나누기

3 NSMC 영화평 20만 편

4 단어구름

5 마무리

공백은 어절을 자른다



어절(語節) — 띄어쓰기로 갈린 덩어리

공백이 자르는 것은 어절이다. 그런데 어절의 수는 **쓴 사람이 어떻게 띄었느냐**에 달렸다. 이 리뷰를 맞춤법대로 띄면 '어릴 때 보고 지금 다시 봐도 재밌어요ㅋㅋ', 일곱 어절이다.

형태소(形態素) — 뜻을 가진 가장 작은 단위

아래 줄은 오늘 쓸 분석기 Kiwi가 나눈 열두 형태소다. 띄어쓰기와 상관없이 같은 답이 나온다.

단위 셋, 수 셋

단위	무엇으로 가르나	‘영화를 봤다’	
어절	띄어쓰기	영화를 / 봤다	(2)
단어	학교 문법: 조사는 단어, 어미는 아니다	영화 / 를 / 봤다	(3)
형태소	뜻을 가진 가장 작은 단위	영화 / 를 / 보- / -았- / -다	(5)

한국어는 교착어다

어간에 조사와 어미가 겹겹이 붙는다. ‘영화가’, ‘영화를’, ‘영화는’은 어절로는 셋이지만 ‘영화’ 하나에 조사가 붙은 것이다. 공백으로 자르면 이 셋이 **서로 다른 타입**으로 세어진다.

8강에서 이미 본 것

동요 10편은 형태소로 나뉘어 있었다. ‘와서’를 오 아서로 적어 둔 것이 그 흔적이다.
누군가 **손으로** 한 형태소 분석이었다. 오늘은 기계가 한다.

형태소 분석이 하는 일



형태소는 문자열의 조각이 아니다

'와서'를 어디서 잘라도 '오'는 나오지 않는다. 그래서 형태소 분석은 자르기에 그치지 않고 **기본형을 되살린다**. 줄어든 자리(와서 ← 오-+-아서), 받침이 떨어진 자리 (부는 ← 불-), 모양이 바뀐 자리(고마운 ← 고맙-)마다 그렇다.

세 가지 일

가르기 어절 안에서 형태소의 경계를 찾는다

되살리기 바뀐 모양을 사전에 오른 기본형으로 돌린다

품사 붙이기 형태소마다 품사 태그를 단다

겉모습은 하나, 분석은 여럿

아버지가방에들어가신다 — Kiwi는 (가)를 골랐다

(가) 아버지/NNG 가/JKS 방/NNG 에/JKB 들어가/VV …
(나) 아버지/NNG 가방/NNG 에/JKB 들어가/VV …

분석기는 고른다

‘나는’도 그렇다. ‘나는 간다’에서는 나/NP+는/JX이고, ‘하늘을 나는 새’에서는 날-/VV+-는/ETM이다. 분석기는 가능한 분석을 모두 만든 뒤 **앞뒤 맥락**으로 가장 그럴듯한 것을 고른다.

이민철 (2024), 《디지털인문학》 1(1), 109–136

모호한 문장만 모은 평가에서 Kiwi는 평균 **86.7%**를 맞혔고, 기존의 공개 분석기는 50–70%였다고 보고했다.

품사 태그 읽는 법

	태그	뜻
체언	NNG / NNP	일반 명사 / 고유 명사
	NNB / NP	의존 명사 / 대명사
용언	VV / VA	동사 / 형용사
	VX / VCP	보조 용언 / 이다
수식언	MAG / MM	일반 부사 / 관형사
독립언	IC	감탄사

	태그	뜻
조사	JKS / JKO	주격 / 목적격
	JKB / JX	부사격 / 보조사
어미	EP / EF	선어말 / 종결
	EC / ETM	연결 / 관형형 전성
파생	XSV / XR	동사 파생 접미사 / 어근
기호	SF / SW	마침표류 / 기타(ㅋㅋ)

세종 말뭉치의 태그에서 왔다

Kiwi의 태그는 세종 말뭉치의 품사 태그를 바탕으로 한다. 국립국어원 모두의 말뭉치의 형태 분석 말뭉치도 NNG, VV, EF를 그대로 쓴다. 한 번 익히면 계속 쓴다. Kiwi는 용언 태그 뒤에 -I(불규칙 활용)나 -R(규칙 활용)을 덧붙이기도 한다.

출처: https://github.com/bab2min/kiwipiepy (품사 태그)

1 어절과 형태소

2 **Kiwi로 나누기**
첫 분석
기계가 나눈 것

3 NSMC 영화평 20만 편

4 단어구름

5 마무리

Kiwi

Korean Intelligent Word Identifier

이민철이 만들어 공개한 한국어 형태소 분석기다. C++로 짜여 빠르고, `pip` 하나로 깔린다.
파이썬 패키지의 이름은 `kiwipiepy`다.

노트북 입력

```
import kiwipiepy
print(kiwipiepy.__version__)
```

실행 결과

```
0.23.2
```

판본을 적는다

분석기는 판이 바뀌면 결과가 바뀐다. 오늘 슬라이드의 모든 수는 0.23.2로 얻었다.
여러분의 수가 슬라이드와 다르면 판본부터 확인한다. 보고서에도 판본을 적는다.

첫 분석

노트북 입력 — 앞에서 만든 s 그대로

```
for t in kiwi.tokenize(s):
    print(t.form, t.tag)
```

tokenize 문자열을 받아 형태소(Token)의 목록을 돌려준다

form 형태. 되살린 기본형이다

tag 품사 태그

‘어릴’은 어리/VA+ㄹ/ETM, ‘봐도’는 보/VV+어도/EC다. ‘ㅋㅋ’도 기타 기호 SW로 한 자리를 차지한다.

실행 결과

```
어리 VA
ㄹ ETM
때 NNG
보 VV
고 EC
지금 MAG
다시 MAG
보 VV
어도 EC
재밌 VA
어요 EF
ㅋㅋ SW
```

start와 len — 원문의 어디에서 왔나

노트북 입력

```
w = '와서 보니'
for t in kiwi.tokenize(w):
    print(t.form, t.tag, t.start, t.len, w[t.start:t.end])
```

실행 결과

```
오 VV 0 1 와
어서 EC 0 2 와서
보 VV 3 1 보
니 EC 4 1 니
```

두 형태소가 한 글자를 나누어 가진다

오는 '와' 한 글자에서 왔고, 어서는 '와서' 두 글자에 걸쳐 있다. form을 이어 붙여도 원문이 되지 않으니, 원문은 start와 len으로 되짚는다.

8강의 동요는 아서로 적었지만 Kiwi는 '아/어'로 갈리는 어미를 어 쪽으로 모아 적는다.

ᄂᆞ다의 ᄂᆞ은 9강의 그 글자다

노트북 입력

```
ts = kiwi.tokenize('추천한다')
print([t.form for t in ts])
print(ts[2].form == 'ᄂᆞ다')
print(name(ts[2].form[0]))
print(name('ᄂᆞ'))
```

실행 결과

```
['추천', '하', 'ᄂᆞ다']
False
HANGUL JONGSEONG NIEUN
HANGUL LETTER NIEUN
```

받침은 종성 자모로 적는다

Kiwi는 어미의 첫머리에 오는 받침 ㄴ·ㄹ을 **종성 자모**로 적는다. 9강의 NFD 슬라이드에 나온 바로 그 U+11AB다. 키보드로 친 ㄴ은 **호환 자모** U+3134라서 ==가 False가 된다.

어미로 검색할 때 조심한다. `kiwi.tokenize(…, compatible_jamo=True)`로 받으면 호환 자모가 나온다.

사람이 나눈 것과 기계가 나눈 것

노트북 입력 — 8강 동요 [7]행의 원문과, 같은 '부는'이 든 다른 문장

```
song = '산 위에서 부는 바람 시원한 바람 그 바람은 좋은 바람 고마운 바람'
print(' '.join(t.form for t in kiwi.tokenize(song)))
print(' '.join(t.form for t in kiwi.tokenize('바람이 부는 날')))
```

실행 결과와 8강의 코퍼스

```
산 위 에서 부 는 바람 시원 하 ᆫ 바람 그 바람 은 좋 은 바람 고맙 은 바람
바람 이 불 는 날
```

```
산 위 에서 불 는 바람 시원하 은 바람 그 바람 은 좋 은 바람 고맙 은 바람
```

오류 '부는'을 명사 부로 봤다. '바람이 부는 날'에서는 바르게 불로 되살렸다

깊이 '시원한'을 어근 시원과 접미사 하로 한 번 더 쪼갰다

표기 관형형 어미를 사람은 은으로, Kiwi는 모음 뒤에서 ᆫ으로 적었다

띄어쓰기가 무너진 리뷰

노트북 입력 — NSMC 학습용 파일의 셋째 리뷰

```
r = '너무재밌었다그래서보는것을추천한다'
print(' '.join(f'{t.form}/{t.tag}' for t in kiwi.tokenize(r)))
```

실행 결과, 그리고 같은 리뷰를 KoNLPy의 두 분석기로 (비교용. 설치하지 않는다)

```
너무/MAG 재/XPN 밌/NNG 었/EP 다/EF 그래서/MAJ 보/VV 는/ETM 것/NNB 을/JKO 추천/NNG 하/XSV
↪  ㄴ다/EF
```

```
Okt      너/Modifier 무재/Noun 밌었/Noun 다그/Noun 래서/Noun 보는것을/Verb 추천/Noun
↪  한/Josa 다/Adverb
Komoran  너무재밌었다그래서보는것을추천한다/NA
```

Kiwi는 띄어쓰기가 없어도 경계를 찾았지만 오타 '재밌'은 재+밌으로 잘랐다. **분석기는 모두 틀린다. 틀리는 곳이 다를 뿐이다.**

확인 질문

여기까지 정리

1. ‘영화를 봤다’는 몇 어절이고 몇 형태소인가?
2. 형태소 분석이 “자르기”로 끝나지 않는 까닭을 ‘와서’로 설명해 보자.
3. 고맙/VA-I의 -I는 무엇을 뜻하는가? 8강 동요의 고맙 은과 이어 보자.
4. `kiwi.tokenize('추천한다')[2].form == '느다'`가 `False`인 까닭은?
5. 동요 [7]행에서 Kiwi가 사람과 다르게 나눈 세 곳은 각각 오류인가, 규칙의 차이인가?

1 어절과 형태소

2 Kiwi로 나누기

3 **NSMC 영화평 20만 편**
데이터 읽기
어절로 세기와 형태소로 세기

4 단어구름

5 마무리

NSMC

Naver Sentiment Movie Corpus v1.0 (Lucy Park, 2015)

- 네이버 영화의 평점 리뷰 **20만 편**. 평균 35자의 짧은 글이다.
- 평점 1–4점은 **부정**(0), 9–10점은 **긍정**(1)으로 붙였다. 5–8점은 뺐다.
- 부정과 긍정이 10만 편씩이다. 열은 탭으로 나눈 `id`, `document`, `label`

파일 셋

`ratings.txt`	20만 편 전부. 오늘 쓴다
`ratings_train.txt`	그중 15만 편. 19강에서 모델을 학습시킬 때
`ratings_test.txt`	나머지 5만 편. 19강에서 모델을 평가할 때

누구나 자유롭게 쓸 수 있다(CC0 1.0). 출처: `https://github.com/e9t/nsmc`

내려받기

터미널 입력 — 3강에서 요리책을 받던 그 순서

```
mkdir -p data/nsmc
cd data/nsmc
wget https://raw.githubusercontent.com/e9t/nsmc/master/ratings.txt
wc -l ratings.txt
head -n 2 ratings.txt
```

실행 결과 (wget의 진행 표시는 뺐다)

```
200001 ratings.txt
id      document        label
8112052 어릴때보고 지금다시봐도 재밌어요ㅋㅋ    1
```

첫 행이 머리글이니 리뷰는 200,000편이고, 그 첫 편이 오늘의 첫 문장이었다. `data/`는 3강의 `.gitignore`에 걸려 커밋되지 않는다.

9강이 약속한 첫 줄 — encoding='utf-8'

노트북 입력

```
df = pd.read_csv('data/nsmc/ratings.txt', sep='\t', encoding='utf-8')
print(df.shape)
print(df.isna().sum())
```

실행 결과

```
(200000, 3)
id           0
document     8
label        0
dtype: int64
```

7강에서 배운 대로 빈 자리부터 센다

20만 행이 제대로 들어왔다. 그런데 document가 빈 리뷰가 **여덟 편** 있다. 파일을 열 때 encoding=을 적는 것, 읽자마자 빈 자리를 세는 것. 둘 다 습관으로 삼는다.

9강이 약속한 둘째 줄 — NFC

노트북 입력

```
df = df.dropna()
nfc = df['document'].str.normalize('NFC')
print((df['document'] != nfc).sum())
df['document'] = nfc
print(df['label'].value_counts())
```

실행 결과

```
16
label
1    99996
0    99996
Name: count, dtype: int64
```

NFC가 바꾼 16편은 모두 한자였다

‘전율(戰慄)’의 慄이 U+F9D9로 들어 있었다. 본래의 慄은 U+6144다. 옛 완성형은 이 한자를 **독음마다 따로** 넣었다. euc-kr로 D7 C9는 ‘률’ 자리, EB D0는 ‘율’ 자리다. 유니코드는 뒤의 것을 **호환 한자**로 받아 두었고, NFC가 둘을 하나로 모은다.

빈 리뷰 여덟 편을 빼니 부정과 긍정이 99,996편씩 남았다.

공백으로 자르면 '영화'가 흩어진다

노트북 입력

```
ej = df['document'].str.split().explode()
print(len(ej), ej.nunique())
yh = ej[ej.str.startswith('영화')]
print(yh.nunique())
print(yh.value_counts().head(6))
```

실행 결과

```
1518208 449791
6233
document
영화     14472
영화.      4813
영화를     3786
영화는     3289
영화가     3219
영화의     1140
Name: count, dtype: int64
```

split 7강의 .str로 리뷰마다 공백으로 자른다. 칸마다 목록이 든다

explode 목록을 펼쳐 **한 행에 하나씩** 놓는다

'영화'로 시작하는 어절만 **6,233가지**다. '영화.'처럼 문장부호만 붙어도 다른 타입이 된다.

20만 편을 한 번에 나누기

노트북 입력 — 끝날 때까지 기다린다

```
def pos(tokens):
    return [(t.form, t.tag) for t in tokens]

df['morphs'] = [pos(ts) for ts in kiwi.tokenize(df['document'])]
print(df['morphs'].iloc[0][:4])
```

실행 결과 — 첫 리뷰의 앞 네 형태소

```
[('어리', 'VA'), ('ㄹ', 'ETM'), ('때', 'NNG'), ('보', 'VV')]
```

문자열의 목록(여기서는 열 전체)을 넘기면 여러 코어로 나누어 돌고, 리뷰마다 Token의 목록을 돌려준다. 메모리를 아끼려고 함수 pos로 **(형태, 태그) 짝**만 남긴다.

어절과 형태소를 8강의 자로 재면

노트북 입력

```
m = df['morphs'].explode()
print(len(m), m.nunique())
```

실행 결과

```
3948240 58503
```

세는 단위	토큰	타입
어절 (공백으로)	1,518,208	449,791
형태소 (Kiwi로)	3,948,240	58,503

토큰은 2.6배로 늘고, 타입은 7.7분의 1로 줄었다

‘영화가’, ‘영화를’, ‘영화.’가 `('영화', 'NNG')` 하나로 모였다. 타입은 **(형태, 태그) 짝**으로 셌으므로 주격 조사 ‘이’(JKS)와 ‘이다’의 ‘이’(VCP)는 다른 타입이다.

명사만 고르기

노트북 입력

```
def nouns(morphs):
    return [f for f, t in morphs
            if t in ('NNG', 'NNP')]

df['nouns'] = df['morphs'].apply(nouns)
n = df.explode('nouns', ignore_index=True)
print(n['nouns'].value_counts().head(8))
```

실행 결과

```
nouns
영화     78236
연기      9756
최고      8886
평점      8534
생각      7242
스토리     7139
때       7122
드라마     6939
Name: count, dtype: int64
```

`ignore_index=True`는 펼친 행에 색인을 새로 매긴다. 빼면 색인이 겹쳐 뒤의 `pd.crosstab`이 `ValueError`로 멈춘다. 명사가 없는 리뷰 15,933편은 빈 행(NaN)이 되지만 `value_counts`가 건너뛴다.

공백으로 세면 14,472였던 '영화'가 **78,236**이 되었다. 5.4배다.

1 어절과 형태소

2 Kiwi로 나누기

3 NSMC 영화평 20만 편

4 **단어구름**
그리기
구름이 감추는 것
저장소에 남기기

5 마무리

첫 단어구름

노트북 입력 — 마지막 식이 그림이면 노트북이 그대로 보여 준다

```
font = '/usr/share/fonts/truetype/nanum/NanumGothic.ttf'
wc = WordCloud(font_path=font, width=800, height=400,
               background_color='white', random_state=0)
wc.generate_from_frequencies(n['nouns'].value_counts()).to_image()
```



글자 크기가 빈도를 나타낸다. 자리와 색과 방향에는 뜻이 없다.

`random_state=0`을 적었으므로 여러분의 구름도 이것과 같다.

`font_path`에 사전 준비에서 깐 나눔고딕을 주지 않으면 한글이 네모로 나온다.

‘영화’는 리뷰 셋 가운데 하나에 나온다

노트북 입력

```
docfreq = n.groupby('nouns')['id'].nunique()
top = docfreq.nlargest(4)
print((top / len(df)).round(3))
```

실행 결과

```
nouns
영화    0.311
연기    0.044
최고    0.041
평점    0.039
Name: id, dtype: float64
```

8강의 문서 빈도

명사마다 **서로 다른 리뷰 id**가 몇 개인지 센 것이 곧 문서 빈도다. ‘영화’는 리뷰의 31.1%에 나오고, 2위 ‘연기’는 4.4%에 나온다.

어디에나 나오는 말은 리뷰를 가르지 못한다. 20강의 TF-IDF가 이것을 수식으로 만든다.

불용어는 정하는 것이다

불용어(stopword)

세지 않기로 **정한** 말이다. 정답 목록은 없다.

도구가 주는 목록

Kiwi에도 불용어 목록(`kiwipiepy.utils.Stopwords`)이 있다. 100개 가운데 대부분이 조사와 어미이고, 명사는 '때', '말', '사람', '일', '지역' 다섯뿐이다.

오늘 정한 것

명사 가운데 '영화' 하나만 뺀다. 근거는 문서 빈도 31.1%다. '때'와 '말'도 뺄까? 빼도 된다. 다만 **무엇을 뺐는지 적어 둔다**. 8강의 "보고할 때 적어야 할 것"에 한 줄이 늘었다.

부정과 긍정으로 나누어 그리기

노트북 입력

```
stop = {'영화'}
n = n[~n['nouns'].isin(stop)]
for label, side in [(0, 'neg'), (1, 'pos')]:
    vc = n[n['label'] == label]['nouns'].value_counts()
    print(side, list(vc.index[:8]))
    wc.generate_from_frequencies(vc).to_file(f'plots/nsmc-{side}.png')
```

실행 결과

```
neg ['평점', '스토리', '연기', '쓰레기', '내용', '감독', '말', '재미']
pos ['최고', '연기', '감동', '때', '생각', '드라마', '사랑', '평점']
```

~isin 불용어 집합에 **들지 않은** 행만 남긴다. ~는 6강의 "아니다"

to_file 그림 파일로 저장한다. plots/는 7강에서 만들었다

두 구름

부정 (label 0)



긍정 (label 1)



보이는 것

부정 쪽에는 '쓰레기', 긍정 쪽에는 '최고', '감동'이 크다. 여기까지는 짐작대로다. 그런데 부정 쪽 왼쪽 아래의 큰 글자를 보자. **'재미'**다.

어느 쪽에 더 많은가

노트북 입력

```
ct = pd.crosstab(n['nouns'], n['label'])
print(ct.loc[['최고', '감동', '재미',
              '연기', '쓰레기', '최악']])
```

실행 결과

label	0	1
nouns		
최고	640	8246
감동	1349	5143
재미	3569	2089
연기	4419	5337
쓰레기	4363	250
최악	2998	35

‘재미’는 부정 리뷰에 더 많다

pd.crosstab은 두 열의 값을 짝지어 센다. 7강의 groupby를 한 번에 한 셈이다. 두 구름에 다 있던 ‘연기’는 긍정 쪽이 조금 많고, ‘재미’는 **부정 쪽이 1.7배** 많다. 글자 크기는 한 구름 **안에서의** 순서만 보여 주므로, 구름 두 장을 나란히 놓아도 이것은 읽히지 않는다.

부정 리뷰의 '재미'

노트북 입력

```
neg = df[(df['label'] == 0) & df['document'].str.contains('재미')]
print(len(neg))
for d in neg['document'].head(3):
    print(d)
```

실행 결과

```
8065
재미없어 연기 넘 오글거려 ㅋㅋㅋ
전혀 재미도없고 진부한 공포영화
스토리도 그렇고 연기는 볼만한데 너무 재미없어요
```

부정 리뷰의 명사 '재미' 3,569번 뒤에는 '도'(874), '없'(704), '가'(643)가 가장 많이 왔다.
'재미도 없고', '재미 없다'에서 명사만 고르면 **'없다'가 떨어져 나가고 '재미'만 남는다**.

띄어 쓰면 명사가 생긴다

노트북 입력

```
for r in ['재미없다', '재미 없다']:
    print([(t.form, t.tag) for t in kiwi.tokenize(r)])
```

실행 결과

```
[('재미없', 'VA'), ('다', 'EF')]
[('재미', 'NNG'), ('없', 'VA'), ('다', 'EF')]
```

뜻은 같은데 세는 결과가 다르다

띄어쓰기 하나로 명사 '재미'가 생기고 사라진다. 부정 리뷰에 4,207번 나온 형용사 **재미없**은 명사 구름에 처음부터 없다. 긍정 쪽의 **재밌**(10,273번)과 **재미있**(4,628번)은 서로 다른 타입이다.

단어구름을 읽을 때

크기 빈도에 비례하지 않는다. '영화'는 '연기'보다 8.0배 많은데 글자는 1.8배다 (기본값 `relative_scaling=0.5`)

자리 자리와 색과 방향은 무작위다. `random_state`를 적어야 같은 그림이 다시 나온다

맥락 '재미'가 '재미없다'에서 왔는지 그림에는 남지 않는다

비교 구름 두 장의 같은 크기는 같은 빈도가 아니다. 비교는 표로 한다

7강의 원칙과 같다

개수를 견줄 때는 막대그림이 맞는 자리다. 단어구름은 **첫인상**을 얻는 그림이고, 결론은 교차표로 확인한 수로 쓴다. 그림을 그린 뒤에는 반드시 수로 확인한다.

무엇을 남길까

터미널 입력 — 노트북이 목록에 없으면 Ctrl+S로 저장부터 한다

```
cd /workspaces/LDS2026
git status
```

실행 결과 (ratings.txt는 data/ 아래라 3강의 .gitignore에 걸린다)

```
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        10-nsmc.ipynb
        plots/nsmc-neg.png
        plots/nsmc-pos.png

nothing added to commit but untracked files present (use "git add" to track)
```

커밋하기

터미널 입력

```
git add 10-nsmc.ipynb plots/
git commit -m "10강 실습 노트북과 단어구름"
```

실행 결과

```
[main a1b2c3d] 10강 실습 노트북과 단어구름
 3 files changed, 558 insertions(+)
 create mode 100644 10-nsmc.ipynb
 create mode 100644 plots/nsmc-neg.png
 create mode 100644 plots/nsmc-pos.png
```

558줄이 모두 노트북이다. 노트북 안의 첫 구름은 글자로 바뀌어 **한 줄**로 들어 있고, PNG 두 장은 글자가 아니라서 세지 않는다.

푸시하기

터미널 입력

```
git push
```

실행 결과

```
Enumerating objects: 8, done.
Writing objects: 100% (6/6), 599.44 KiB | 17.13 MiB/s, done.
To https://github.com/your-username/LDS2026.git
   e4f5a6b..a1b2c3d  main -> main
```

다시 돌려도 같은 구름

올리기 전에 [Restart] 후 [Run All]로 처음부터 다시 돌려 둔다. `random_state=0`을 적었으므로 다시 돌려도, 남의 컴퓨터에서 돌려도 같은 구름이 나온다.

1 어절과 형태소

2 Kiwi로 나누기

3 NSMC 영화평 20만 편

4 단어구름

5 **마무리**
- 요약
- 다음 시간
- 과제

오늘 쓴 것

하는 일	오늘의 코드
문장을 형태소로	`kiwi.tokenize(s)` → `t.form, t.tag`
원문의 자리	`t.start, t.len`
열 전체를 NFC로	`df['document'].str.normalize('NFC')`
공백으로 자르기	`df['document'].str.split().explode()`
20만 편을 한 번에	`kiwi.tokenize(df['document'])`
목록 펼치기	`df.explode('nouns', ignore_index=True)`
문서 빈도	`n.groupby('nouns')['id'].nunique()`
불용어 빼기	`n[~n['nouns'].isin(stop)]`
두 열 교차표	`pd.crosstab(n['nouns'], n['label'])`
단어구름	`WordCloud(…).generate_from_frequencies(vc)`

형태소 분석기 하나가 더해졌을 뿐, 세고 묶고 그리는 도구는 6·7강의 것 그대로다.

오늘 배운 것

어절과 형태소

- 한 리뷰가 3어절, 12형태소
- 가르고, 되살리고, 품사를 붙인다
- 겉모습 하나에 분석이 여럿

NSMC

- 빈 리뷰 8편, NFC가 고친 16편
- 타입 45만(어절) → 5.9만(형태소)
- '영화'는 리뷰의 31.1%에 나온다

Kiwi

- 세종 계열 태그, 불규칙은 -I
- 받침 어미는 종성 자모 U+11AB
- 사람과 다르게 나누고, 틀리기도 한다

단어구름

- 불용어는 정하고, 적어 둔다
- 크기는 빈도에 비례하지 않는다
- 부정 리뷰의 '재미'는 '재미없다'였다

확인 질문

오늘 것을 정리해 봅시다

1. NSMC를 공백으로 자르면 타입이 449,791, Kiwi로 나누면 58,503이다. 왜 이렇게 차이가 나는가?
2. 형태소의 타입을 형태만이 아니라 (형태, 태그) 짝으로 센 까닭은?
3. `df.explode('nouns')`에 `ignore_index=True`를 빼면 무엇이 문제가 되는가?
4. '영화'를 불용어로 삼은 근거를 8강의 용어로 말해 보자.
5. 부정 리뷰의 단어구름에서 '재미'가 크게 보이는 까닭을 두 가지 들어 보자.

다음 시간에 배울 것 (10월 12일 월요일)

제11강 모두의 말뭉치 (1) JSON 파일 처리

오늘은 누구나 내려받을 수 있는 영화평을 썼다. 다음 시간에는 신청하고 승인을 받아야 쓸 수 있는 국립국어원의 말뭉치를 연다. [숙제07]에서 신청하는 **온라인 대화 말뭉치 2021**이다. 파일은 표가 아니라 JSON이다.

그때 오늘 것을 쓴다

읽자마자 NFC로 맞추기, 목록이 든 열을 `explode`로 펼치기, 무엇을 하나로 셌는지 적어두기. 모두 그대로 쓴다.

10월 14일 수요일은 자율학습일이라 수업이 없다.

과제 안내

[숙제05] 문자 인코딩 연습 [풀이]

마감 **2026-10-08 12:00 (목요일 정오)**. 노트북이 아니라 **서면 풀이 PDF**를 eTL에 낸다.

[숙제06] 언어데이터 관리 사례 발표 논문 선정 [인증]

마감 **2026-10-08 17:00 (목요일 오후 5시)**. 부교재 *The Open Handbook of Linguistic Data Management*의 Part II(14–56장)에서 발표할 장을 1·2·3순위로 골라 장 번호를 낸다. 겹치면 **먼저 낸 사람**이 가져간다.

[숙제07] 모두의 말뭉치 가입 인증 [인증]

마감 **2026-10-08 17:00 (목요일 오후 5시)**. 모두의 말뭉치에 가입해 **온라인 대화 말뭉치 2021**을 신청하고, [말뭉치 신청 내역] 화면을 `hw07-학번.png`로 eTL에 낸다. 승인은 담당자가 하니 미루지 않는다.